Grant-Free Scheduling (GFS) Complete Implementation Manual

OpenAirInterface 5G NR

gNodeB and User Equipment

January 2026

Abstract

This manual provides comprehensive documentation for the Grant-Free Scheduling implementation in OpenAirInterface 5G NR, covering both the gNodeB (gNB) and User Equipment (UE) sides. Grant-Free scheduling enables User Equipment to transmit uplink data on pre-configured resources without waiting for explicit scheduling grants from the gNB, achieving significant latency reduction for URLLC and time-critical applications.

This document covers the complete architecture, data structures, APIs, integration procedures, and testing guidelines for deploying Grant-Free scheduling in OAI 5G NR systems.

Contents

I Overview and Architecture **5**

1 Introduction **6**
1.1 Purpose . . . 6
1.2 Key Benefits . . . 6
1.3 Traditional vs Grant-Free Flow . . . 6
1.3.1 Traditional Scheduling . . . 6
1.3.2 Grant-Free Scheduling . . . 7
1.4 Features . . . 7

2 System Architecture **8**
2.1 End-to-End Architecture . . . 8
2.2 gNB Architecture Detail . . . 9
2.3 UE Architecture Detail . . . 9
2.4 Design Principles . . . 9

3 Configuration Matching Requirements **11**
3.1 Critical Requirement . . . 11
3.2 Parameters That Must Match . . . 11
3.3 GF Occasion Formula . . . 12
3.3.1 Example: periodicity=4, offset=1 . . . 12

II gNodeB Implementation **13**

4 gNB Data Structures **14**
4.1 Per-UE Configuration Structure . . . 14
4.2 Grant-Free Manager Structure . . . 15

5 gNB API Reference **16**
5.1 Initialization . . . 16
5.2 Configuration Functions . . . 16
5.3 Slot Processing Functions . . . 16

6 gNB Slot Processing **18**
6.1 Processing Flow . . . 18
6.2 GF Occasion Check Implementation . . . 18

7 gNB HARQ Management **20**
7.1 HARQ Process Reservation . . . 20
7.2 NDI Tracking . . . 20
7.3 RV Sequence . . . 20

III UE Implementation 21

8 UE Data Structures 22
8.1 GF Configuration Structure . . . 22
8.2 No-Data Behavior Enum . . . 23

9 UE API Reference 24
9.1 Configuration Functions . . . 24
9.2 Slot Processing Functions . . . 24

10 UE Slot Processing 26
10.1 Processing Flow . . . 26
10.2 GF Occasion Check Implementation . . . 26

11 UE MAC PDU Construction 28
11.1 MAC PDU Structure for GF . . . 28
11.2 MAC Subheader Formats . . . 28
11.2.1 Fixed Subheader (1 byte) . . . 28
11.2.2 Short Subheader (2 bytes) . . . 28
11.2.3 Long Subheader (3 bytes) . . . 28
11.3 MAC PDU Building Implementation . . . 29
11.4 Important MAC PDU Rules . . . 30

12 UE HARQ Management 31
12.1 HARQ Process Handling . . . 31

IV Integration and Deployment 33

13 File Structure 34
13.1 gNB Files . . . 34
13.2 UE Files . . . 34

14 Build Integration 35
14.1 gNB Configuration . . . 35
14.2 Build Commands for UE . . . 35

15 Configuration Parameters 36
15.1 Default Configuration . . . 36
15.2 Configuration Validation . . . 36

16 Known Limitations and Current problems 38
16.1 Current Limitations . . . 38
16.2 Current Problems . . . 38
16.2.1 USRP Operating on USB 2.0 Instead of USB 3.0 . . . 38
16.2.2 Invalid MAC PDU Parsing Error . . . 39
16.3 Problem Analysis . . . 39
16.3.1 Original Problematic Implementation . . . 39
16.3.2 Issues Identified . . . 40
16.4 OAI Standard Approach . . . 40
16.4.1 How Normal OAI Handles MAC PDU Transmission . . . 40
16.4.2 Key OAI Principles . . . 40
16.5 Corrected Implementation . . . 41

16.5.1 Corrected MAC PDU Attachment 41
16.5.2 Release ul_config 42
16.6 Expected Results 42
16.6.1 Resolution of Issues 42

Part I

Overview and Architecture

Chapter 1

Introduction

1.1 Purpose

This manual documents the complete Grant-Free (GF) scheduling implementation for OpenAir-Interface 5G NR, covering both the gNodeB (gNB) and User Equipment (UE) sides. Grant-Free scheduling enables UEs to transmit uplink data on pre-configured resources without waiting for explicit scheduling grants, achieving significant latency reduction for URLLC and time-critical applications.

1.2 Key Benefits

Table 1.1 compares traditional scheduling with Grant-Free scheduling.

Table 1.1: Latency Comparison: Traditional vs Grant-Free Scheduling

Metric	**Traditional Scheduling**	**Grant-Free Scheduling**
UL Latency	8–12 ms	< 1 ms
Signaling Overhead	High (SR $\rightarrow$ BSR $\rightarrow$ Grant $\rightarrow$ Data)	Minimal
Suitable For	eMBB, best-effort traffic	URLLC, IoT, industrial automation

1.3 Traditional vs Grant-Free Flow

1.3.1 Traditional Scheduling

In traditional scheduling, the UE must request resources before transmitting data:

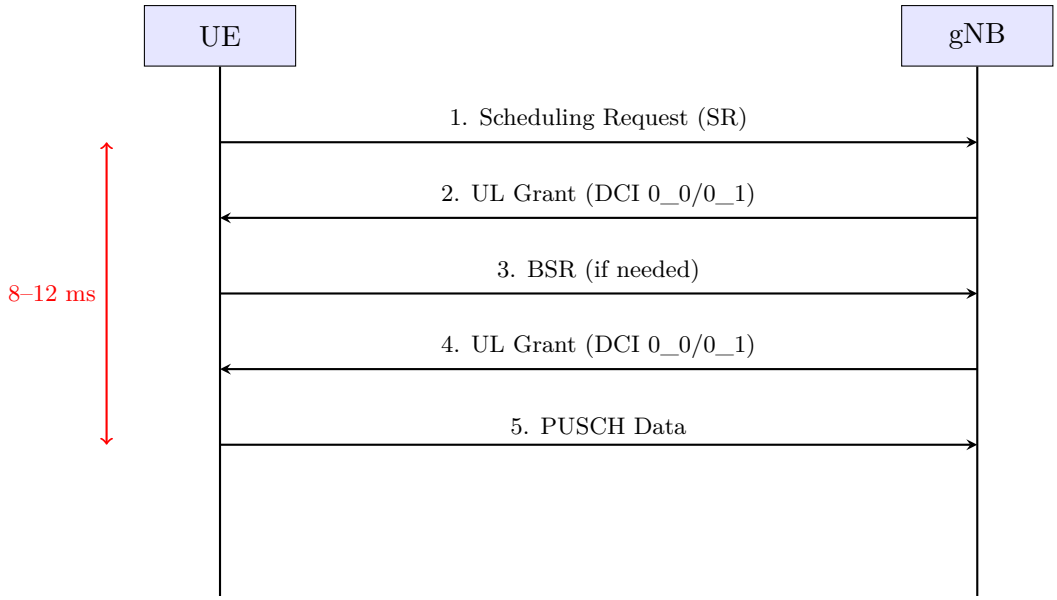


Figure 1.1: Traditional Scheduling Message Flow

1.3.2 Grant-Free Scheduling

With Grant-Free scheduling, the UE transmits directly on pre-configured resources:

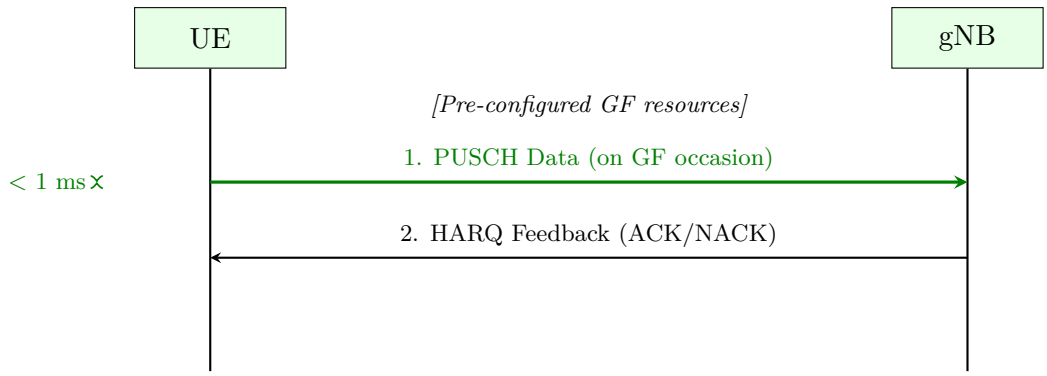


Figure 1.2: Grant-Free Scheduling Message Flow

1.4 Features

The Grant-Free scheduling implementation provides the following features:

- **Autonomous UE Transmission:** UEs transmit on pre-configured resources without DCI grants
- **Configurable Periodicity:** GF occasions every 1 to 65535 slots
- **Multiple UE Support:** Up to 32 UEs with Grant-Free configurations (gNB)
- **Multiple Configs per UE:** Up to 4 GF configurations per UE
- **Full HARQ Support:** ACK/NACK feedback with retransmission handling
- **Coexistence:** Runs alongside normal Proportional Fair scheduling
- **Standards-Based:** Follows 3GPP TS 38.321, 38.214, 38.211 principles

Chapter 2

System Architecture

2.1 End-to-End Architecture

Figure 2.1 shows the complete end-to-end architecture of the Grant-Free scheduling implementation.

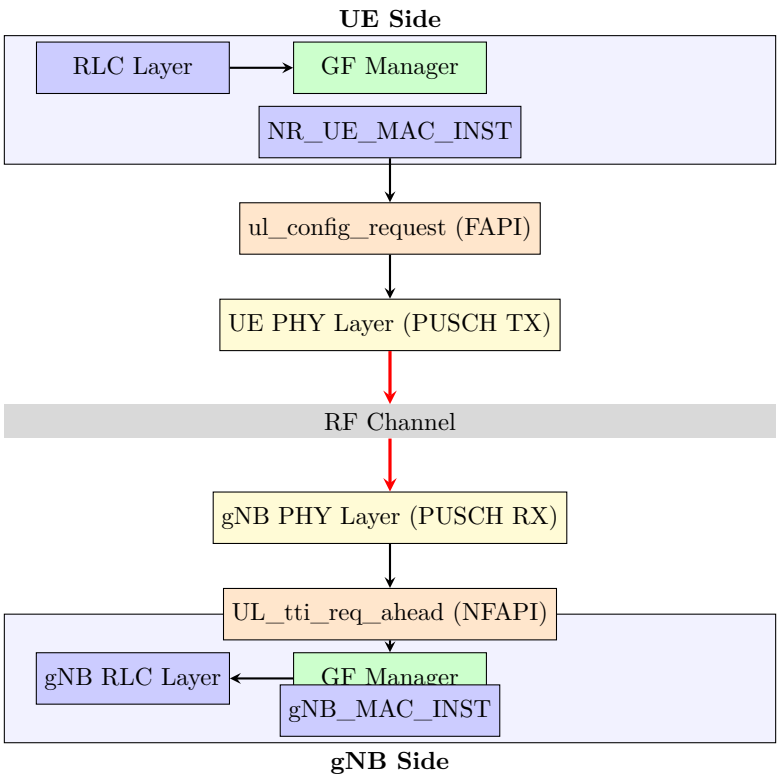


Figure 2.1: End-to-End Grant-Free Architecture

2.2 gNB Architecture Detail

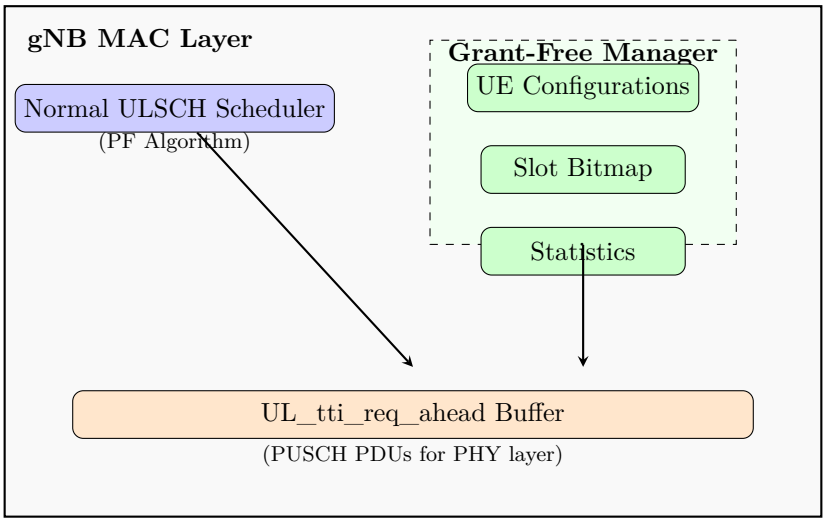


Figure 2.2: gNB Grant-Free Manager Architecture

2.3 UE Architecture Detail

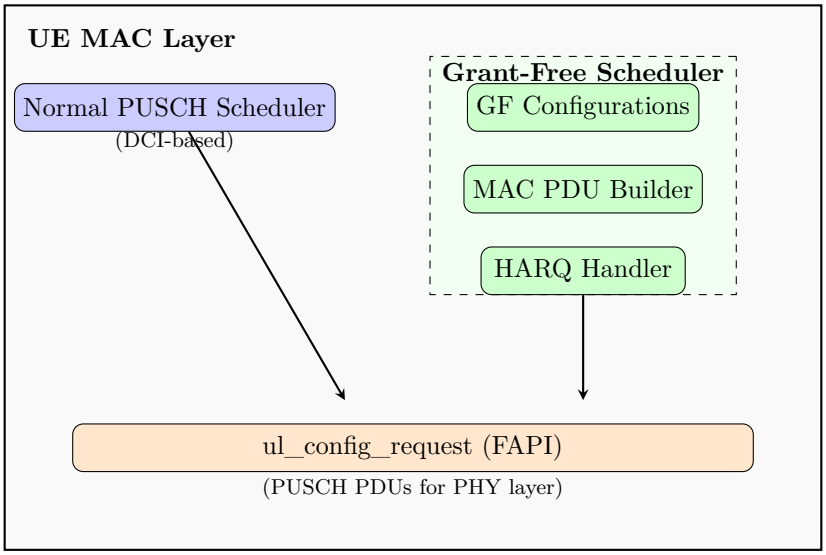


Figure 2.3: UE Grant-Free Scheduler Architecture

2.4 Design Principles

The implementation follows these key design principles:

1. **Grant-Free First:** GF resources are checked/reserved before normal scheduling runs
2. **Non-Exclusive Operation:** Both GF and normal schedulers run concurrently
3. **Configuration Synchronization:** UE and gNB configurations must match exactly
4. **FAPI/NFAPI Compliance:** Uses standard interface structures
5. **Minimal Core Changes:** Integrates via hooks in existing scheduler

6. **Delayed Activation:** GF activation delayed ∼50 slots after connection to allow RRC completion

Chapter 3

Configuration Matching Requirements

3.1 Critical Requirement

Critical Warning

The UE and gNB must have identical GF configurations for successful operation.
Mismatched configurations will result in:

- Failed PUSCH reception at gNB
- CRC errors
- DTX (no detection)
- HARQ failures

3.2 Parameters That Must Match

Table 3.1 lists all parameters that must be identical on both UE and gNB.

Table 3.1: Configuration Parameters That Must Match

Parameter	UE Field	gNB Field	Notes
Periodicity	`gf_config.periodicity`	`gf_ue_config.periodicity`	Exact match
Offset	`gf_config.offset`	`gf_ue_config.offset`	Exact match
Start Symbol	`gf_config.start_symbol`	`gf_ue_config.start_symbol`	Exact match
Nr of Symbols	`gf_config.nr_of_symbols`	`gf_ue_config.nr_of_symbols`	Exact match
RB Start	`gf_config.rb_start`	`gf_ue_config.rb_start`	Exact match
RB Size	`gf_config.rb_size`	`gf_ue_config.rb_size`	Exact match
MCS	`gf_config.mcs`	`gf_ue_config.mcs`	Exact match
MCS Table	`gf_config.mcs_table`	`gf_ue_config.mcs_table`	Exact match
HARQ Process	`gf_config.harq_process_id`	`gf_ue_config.harq_process_id`	Exact match
DMRS Type	`gf_config.dmrs_config_type`	`gf_ue_config.dmrs_config_type`	Exact match
DMRS Ports	`gf_config.dmrs_ports`	`gf_ue_config.dmrs_ports`	Exact match
RNTI	`gf_config.rnti` or `mac->crnti`	`gf_ue_config.rnti`	Exact match

3.3 GF Occasion Formula

Both UE and gNB use the same formula to determine GF occasions:

$$\text{IS_GF_OCCASION} = (\text{absolute_slot} \geq \text{offset}) \wedge ((\text{absolute_slot} - \text{offset}) \mod \text{periodicity} = 0) \tag{3.1}$$

where:

$$\text{absolute_slot} = \text{frame} \times \text{slots_per_frame} + \text{slot} \tag{3.2}$$

3.3.1 Example: periodicity=4, offset=1

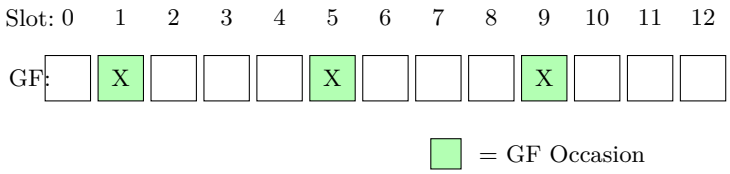


Figure 3.1: GF Occasion Pattern with periodicity=4, offset=1

Part II

gNodeB Implementation

Chapter 4

gNB Data Structures

4.1 Per-UE Configuration Structure

The `nr_gf_gnb_ue_config_t` structure stores the Grant-Free configuration for each UE. It is defined in `openair2/LAYER2/NR_MAC_gNB/nr_mac_gNB.h`.

```
typedef struct nr_gf_gnb_ue_config {
  // State
  bool enabled;                     // GF enabled for this UE
  bool active;                      // GF currently active

  // UE Identification
  rnti_t rnti;                      // UE RNTI
  int ue_id;                        // Internal UE ID

  // Time Domain Allocation
  uint16_t periodicity;             // Period in slots (1-65535)
  uint16_t offset;                  // Slot offset (0 to periodicity-1)
  uint8_t start_symbol;             // Starting OFDM symbol (0-13)
  uint8_t nr_of_symbols;            // Number of symbols (1-14)
  uint8_t mapping_type;             // Type A (0) or Type B (1)

  // Frequency Domain Allocation
  uint16_t rb_start;                // Starting PRB in BWP
  uint16_t rb_size;                 // Number of PRBs
  uint8_t frequency_hopping;        // 0 = disabled

  // Modulation & Coding
  uint8_t mcs;                      // MCS index (0-28)
  uint8_t mcs_table;                // MCS table (0, 1, or 2)

  // HARQ Configuration
  uint8_t harq_process_id;          // Dedicated HARQ process (0-15)
  uint8_t rv_sequence[4];           // RV sequence {0, 2, 3, 1}
  uint8_t expected_ndi;             // Expected NDI
  uint8_t current_round;            // Current retransmission round

  // DMRS Configuration
  uint8_t dmrs_config_type;         // Type 1 (0) or Type 2 (1)
  uint8_t dmrs_ports;               // DMRS port bitmap
  uint16_t dmrs_scrambling_id;      // Scrambling ID (0 = use PCI)
  uint8_t num_dmrs_cdm_grps_no_data;

  // Pre-computed PUSCH PDU (for efficiency)
  nfapi_nr_pusch_pdu_t pusch_pdu;

```

```
  // Statistics
  uint32_t rx_count;                // Total receptions
  uint32_t rx_success;              // Successful (CRC OK)
  uint32_t rx_failure;              // Failed (CRC error)
  uint64_t total_bytes_received;
  uint32_t harq_ack_sent;
  uint32_t harq_nack_sent;
  int8_t last_sinr_db;
} nr_gf_gnb_ue_config_t;
```

Listing 4.1: Per-UE GF Configuration Structure (gNB)

4.2 Grant-Free Manager Structure

The `nr_gf_gnb_manager_t` structure manages all Grant-Free configurations at the gNB.

```
#define NR_GF_MAX_UES 32
#define NR_GF_MAX_CONFIGS_PER_UE 4

typedef struct nr_gf_gnb_manager {
  bool enabled;                     // GF globally enabled

  // UE configurations array
  nr_gf_gnb_ue_config_t ue_configs[NR_GF_MAX_UES][NR_GF_MAX_CONFIGS_PER_UE];
  uint8_t num_configured_ues;

  // Slot bitmap for quick GF occasion lookup (O(1))
  uint64_t slot_bitmap[1024];       // Covers 1024 slots

  // HARQ process reservation bitmap
  uint16_t gf_harq_process_bitmap; // Bit set = process reserved for GF

  // Global statistics
  uint64_t total_gf_occasions;
  uint64_t total_gf_receptions;
  uint64_t total_gf_successes;
  uint64_t total_gf_collisions;
} nr_gf_gnb_manager_t;
```

Listing 4.2: Grant-Free Manager Structure (gNB)

Chapter 5

gNB API Reference

5.1 Initialization

```c
/**
 * Initialize Grant-Free manager
 * Called during gNB MAC initialization
 */
void nr_gf_gnb_init(gNB_MAC_INST *gNB);
```

Listing 5.1: GF Initialization Function

5.2 Configuration Functions

```c
/**
 * Configure GF for a specific UE
 * @param gNB      gNB MAC instance
 * @param ue_id    Internal UE ID
 * @param rnti     UE RNTI
 * @param gf_config Configuration parameters
 * @return 0 on success, -1 on error
 */
int nr_gf_gnb_configure_ue(gNB_MAC_INST *gNB,
                           int ue_id,
                           rnti_t rnti,
                           nr_gf_gnb_ue_config_t *gf_config);

/**
 * Configure default GF parameters for testing
 */
void nr_gf_gnb_configure_default(gNB_MAC_INST *gNB,
                                 int ue_id,
                                 rnti_t rnti);

/**
 * Release GF configuration for a UE
 */
void nr_gf_gnb_release_ue(gNB_MAC_INST *gNB,
                          int ue_id,
                          int config_index);
```

Listing 5.2: GF Configuration Functions (gNB)

5.3 Slot Processing Functions

```c
/**
 * Main GF slot processing function
 * Called at start of each slot before normal ULSCH scheduling
 */
void nr_gf_gnb_slot_process(gNB_MAC_INST *gNB,
                            frame_t frame,
                            int slot);

/**
 * Check if any GF occasions exist in this slot
 * Returns slot info with list of expected UEs
 */
nr_gf_gnb_slot_info_t* nr_gf_gnb_check_occasions(gNB_MAC_INST *gNB,
                                                 frame_t frame,
                                                 int slot);

/**
 * Schedule PUSCH reception for GF UEs
 */
void nr_gf_gnb_schedule_reception(gNB_MAC_INST *gNB,
                                  nr_gf_gnb_slot_info_t *slot_info,
                                  frame_t frame,
                                  int slot);
```

Listing 5.3: GF Slot Processing Functions (gNB)

Chapter 6

gNB Slot Processing

6.1 Processing Flow

Figure 6.1 shows the gNB slot processing flow with Grant-Free integration.

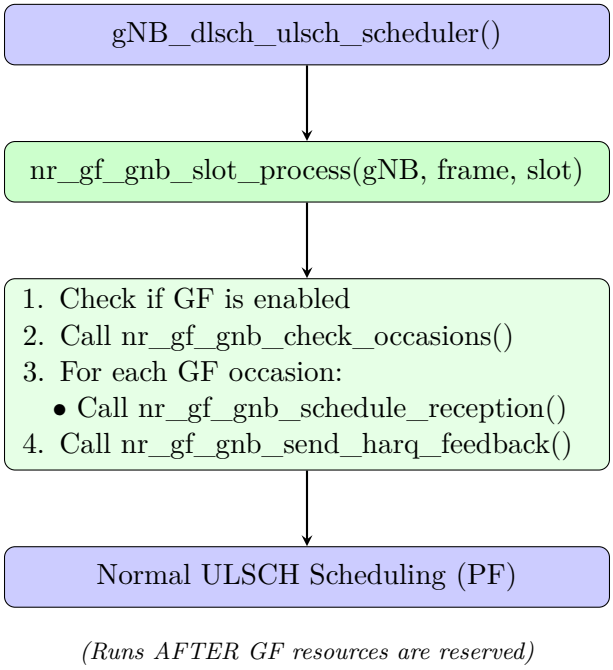


Figure 6.1: gNB Slot Processing Flow

6.2 GF Occasion Check Implementation

```
nr_gf_gnb_slot_info_t* nr_gf_gnb_check_occasions(gNB_MAC_INST *gNB,
                                                 frame_t frame,
                                                 int slot) {
  nr_gf_gnb_manager_t *gf = &gNB->gf_manager;
  static nr_gf_gnb_slot_info_t slot_info;

  slot_info.num_expected_ues = 0;

  int slots_per_frame = gNB->frame_structure.numb_slots_frame;
  int absolute_slot = frame * slots_per_frame + slot;

```

```
  // Check each configured UE
  for (int ue = 0; ue < NR_GF_MAX_UES; ue++) {
    for (int cfg = 0; cfg < NR_GF_MAX_CONFIGS_PER_UE; cfg++) {
      nr_gf_gnb_ue_config_t *ue_cfg = &gf->ue_configs[ue][cfg];

      if (!ue_cfg->enabled || !ue_cfg->active)
        continue;

      // Check GF occasion formula
      int slot_diff = absolute_slot - ue_cfg->offset;
      if (slot_diff >= 0 && (slot_diff % ue_cfg->periodicity) == 0) {
        // This is a GF occasion for this UE
        slot_info.ue_configs[slot_info.num_expected_ues++] = ue_cfg;
      }
    }
  }

  return &slot_info;
}
```

Listing 6.1: GF Occasion Check Implementation (gNB)

Chapter 7

gNB HARQ Management

7.1 HARQ Process Reservation

GF uses dedicated HARQ processes to avoid conflicts with dynamic scheduling:

```
// Reserve HARQ process for GF
gf_manager->gf_harq_process_bitmap |= (1 << harq_process_id);

// Check if HARQ process is reserved for GF
bool is_gf_harq = (gf_manager->gf_harq_process_bitmap >> harq_pid) & 1;
```

Listing 7.1: HARQ Process Reservation

7.2 NDI Tracking

```
// On successful reception (ACK)
gf_cfg->expected_ndi = !gf_cfg->expected_ndi;  // Toggle
gf_cfg->current_round = 0;

// On failed reception (NACK)
gf_cfg->current_round++;
if (gf_cfg->current_round >= 4) {
  // Max retransmissions - reset
  gf_cfg->expected_ndi = !gf_cfg->expected_ndi;
  gf_cfg->current_round = 0;
}
```

Listing 7.2: NDI Tracking for HARQ

7.3 RV Sequence

The standard 3GPP RV sequence is: $\{0, 2, 3, 1\}$

```
uint8_t rv = gf_cfg->rv_sequence[gf_cfg->current_round % 4];
```

Listing 7.3: RV Selection

Part III

UE Implementation

Chapter 8

UE Data Structures

8.1 GF Configuration Structure

The `nr_gf_config_t` structure stores the Grant-Free configuration on the UE side. It is defined in `openair2/LAYER2/NR_MAC_UE/mac_defs.h`.

```c
#define NR_MAX_GF_CONFIGS 4

typedef struct nr_gf_config {
  // State
  bool enabled;           // Configuration is valid
  bool active;            // Currently active

  // Time Domain
  uint16_t periodicity;        // Period in slots (1-65535)
  uint16_t offset;             // Slot offset (0 to periodicity-1)
  uint8_t start_symbol;        // Starting OFDM symbol (0-13)
  uint8_t nr_of_symbols;       // Number of symbols (1-14)
  uint8_t mapping_type;        // Type A (0) or Type B (1)
  uint8_t time_domain_allocation;  // TDA index (for reference)

  // Frequency Domain
  uint16_t rb_start;           // Starting PRB
  uint16_t rb_size;            // Number of PRBs
  uint8_t frequency_hopping;   // 0=disabled, 1=intra-slot, 2=inter-slot

  // MCS
  uint8_t mcs;                 // MCS index (0-28)
  uint8_t mcs_table;           // MCS table (0, 1, or 2)

  // HARQ
  uint8_t harq_process_id;     // HARQ process (0-15)
  uint8_t rv_sequence[4];      // RV sequence {0, 2, 3, 1}
  uint8_t ndi_toggle;          // Current NDI state
  uint8_t max_retransmissions; // Max HARQ retransmissions
  uint8_t current_rv_index;    // Current position in RV sequence
  uint8_t repK;                // Repetitions (1, 2, 4, 8)

  // DMRS
  uint8_t dmrs_config_type;    // Type 1 (0) or Type 2 (1)
  uint8_t dmrs_ports;          // Port bitmap
  uint16_t dmrs_scrambling_id; // Scrambling ID (0 = use PCI)
  uint8_t num_dmrs_cdm_grps_no_data;
  uint8_t dmrs_add_pos;        // Additional DMRS positions

  // Power Control
```

```
  int8_t delta_power;          // Power offset relative to normal PUSCH
  bool transform_precoding;    // DFT-s-OFDM enable

  // Configuration
  uint8_t config_index;        // Index in array
  uint16_t rnti;               // RNTI (0 = use C-RNTI)

  // Behavior
  uint8_t no_data_behavior;    // GF_NO_DATA_SKIP, etc.
  bool include_bsr_always;     // Always include BSR

  // Statistics
  uint32_t tx_count;
  uint32_t tx_with_data;
  uint32_t tx_empty;
  uint32_t harq_nack_count;
  uint32_t harq_dtx_count;
  uint32_t successful_tx_count;
  uint32_t failed_tx_count;
  uint32_t total_bytes_transmitted;
  uint32_t total_bsr_sent;
  uint32_t last_tbs;
  uint8_t last_mcs_used;
  frame_t last_tx_frame;
  int last_tx_slot;
} nr_gf_config_t;
```

Listing 8.1: GF Configuration Structure (UE)

8.2 No-Data Behavior Enum

```
typedef enum {
  GF_NO_DATA_SKIP = 0,        // Don't transmit if no data
  GF_NO_DATA_SEND_BSR = 1,    // Send BSR only
  GF_NO_DATA_SEND_PADDING = 2 // Send padding
} gf_no_data_behavior_t;

#define GF_NO_DATA_DEFAULT_BEHAVIOR GF_NO_DATA_SKIP
```

Listing 8.2: No-Data Behavior Options

Chapter 9

UE API Reference

9.1 Configuration Functions

```c
/**
 * Configure Grant-Free parameters
 * @param mac       UE MAC instance
 * @param gf_params Configuration parameters
 * @return 0 on success, -1 on error
 */
int nr_ue_configure_grant_free(NR_UE_MAC_INST_t *mac,
                               nr_gf_config_t *gf_params);

/**
 * Initialize default GF configuration for testing
 * Should only be called when UE is connected
 */
void nr_ue_init_default_grant_free(NR_UE_MAC_INST_t *mac);

/**
 * Release a GF configuration
 * @param config_index Index of configuration to release
 */
int nr_ue_release_grant_free(NR_UE_MAC_INST_t *mac,
                             uint8_t config_index);
```

Listing 9.1: UE Configuration Functions

9.2 Slot Processing Functions

```c
/**
 * Check if current slot is a GF occasion
 * @return Pointer to matching GF config, or NULL
 */
nr_gf_config_t* nr_ue_check_grant_free_occasion(NR_UE_MAC_INST_t *mac,
                                                frame_t frame,
                                                int slot);

/**
 * Main GF scheduling function
 * Called from nr_ue_ul_scheduler()
 */
void nr_ue_schedule_grant_free(NR_UE_MAC_INST_t *mac,
                               frame_t frame,
                               int slot);

```

```c
/**
 * Configure PUSCH PDU using GF parameters
 */
int nr_config_pusch_pdu_grant_free(NR_UE_MAC_INST_t *mac,
                                   nr_gf_config_t *gf,
                                   nfapi_nr_ue_pusch_pdu_t *pusch_pdu,
                                   frame_t frame,
                                   int slot);
```

Listing 9.2: UE Slot Processing Functions

Chapter 10

UE Slot Processing

10.1 Processing Flow

Figure 10.1 shows the UE slot processing flow with Grant-Free integration.

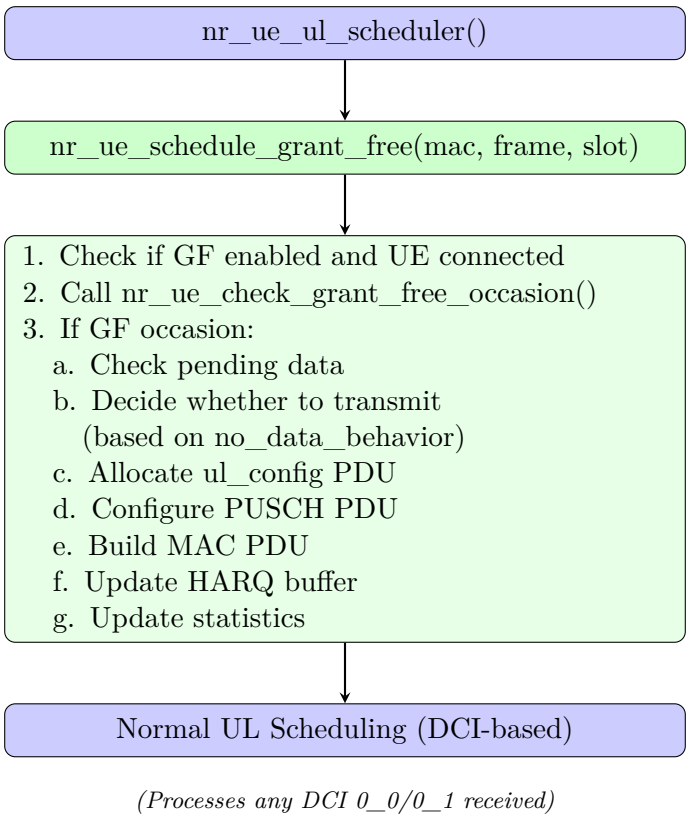


Figure 10.1: UE Slot Processing Flow

10.2 GF Occasion Check Implementation

```
nr_gf_config_t* nr_ue_check_grant_free_occasion(NR_UE_MAC_INST_t *mac,
                                                frame_t frame,
                                                int slot) {
  if (!mac->gf_enabled || mac->num_gf_configs == 0)
    return NULL;

```

```
  int n_slots_frame = mac->frame_structure.numb_slots_frame;
  int absolute_slot = frame * n_slots_frame + slot;

  for (int i = 0; i < mac->num_gf_configs; i++) {
    nr_gf_config_t *gf = &mac->gf_config[i];

    if (!gf->enabled || !gf->active)
      continue;

    if (gf->periodicity == 0)
      continue;

    int slot_diff = absolute_slot - gf->offset;
    if (slot_diff < 0)
      continue;

    if ((slot_diff % gf->periodicity) == 0) {
      // Verify it's an UL slot
      if (!is_ul_slot(slot, &mac->frame_structure))
        continue;

      return gf;
    }
  }

  return NULL;
}
```

Listing 10.1: GF Occasion Check Implementation (UE)

Chapter 11

UE MAC PDU Construction

11.1 MAC PDU Structure for GF

Figure 11.1 shows the structure of a MAC PDU for Grant-Free transmission.

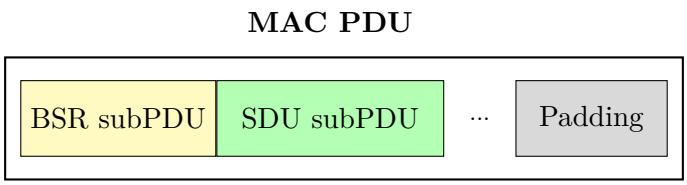


Figure 11.1: MAC PDU Structure for Grant-Free Transmission

11.2 MAC Subheader Formats

11.2.1 Fixed Subheader (1 byte)

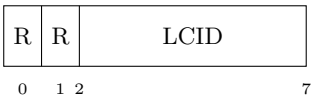


Figure 11.2: Fixed Subheader Format

11.2.2 Short Subheader (2 bytes)

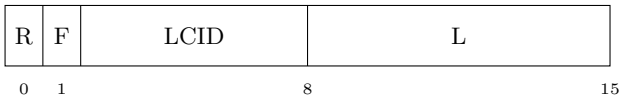


Figure 11.3: Short Subheader Format

11.2.3 Long Subheader (3 bytes)

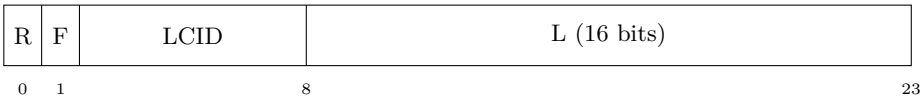


Figure 11.4: Long Subheader Format

11.3 MAC PDU Building Implementation

```c
int nr_ue_gf_build_mac_pdu(NR_UE_MAC_INST_t *mac,
                           nr_gf_config_t *gf,
                           uint8_t *pdu,
                           uint32_t pdu_size) {
  uint8_t *pdu_ptr = pdu;
  uint8_t *pdu_end = pdu + pdu_size;

  // ========== Add BSR if triggered ==========
  if (mac->scheduling_info.BSR_reporting_active) {
    NR_BSR_SHORT short_bsr = {/* ... calculate ... */};

    // Add BSR subheader (fixed, 1 byte)
    *(NR_MAC_SUBHEADER_FIXED *)pdu_ptr = (NR_MAC_SUBHEADER_FIXED){
      .R = 0,
      .LCID = UL_SCH_LCID_S_BSR  // 0x3D
    };
    pdu_ptr += sizeof(NR_MAC_SUBHEADER_FIXED);

    // Add BSR payload (1 byte)
    *(NR_BSR_SHORT *)pdu_ptr = short_bsr;
    pdu_ptr += sizeof(NR_BSR_SHORT);

    mac->scheduling_info.BSR_reporting_active = NR_BSR_TRIGGER_NONE;
  }

  // ========== Add SDUs from RLC ==========
  for (int idx = 0; idx < mac->lc_ordered_list.count; idx++) {
    // ... get SDU from RLC and add with appropriate subheader ...
  }

  // ========== Add Padding ==========
  int remaining = pdu_end - pdu_ptr;
  if (remaining > 0) {
    // Single padding subheader
    *(uint8_t *)pdu_ptr = UL_SCH_LCID_PADDING;  // 0x3F
    pdu_ptr++;

    // Fill rest with zeros
    if (remaining > 1)
      memset(pdu_ptr, 0x00, remaining - 1);
  }

  return pdu_ptr - pdu;
}
```

Listing 11.1: MAC PDU Building (Simplified)

11.4 Important MAC PDU Rules

MAC PDU Construction Rules

1. **Use OAI structures:** Always use `NR_MAC_SUBHEADER_FIXED`, `NR_MAC_SUBHEADER_SHORT`, `NR_MAC_SUBHEADER_LONG`, `NR_BSR_SHORT`
2. **Network byte order:** 16-bit length fields use `htons()` for network byte order
3. **Single padding subheader:** Only one padding LCID (0x3F), rest is filler bytes
4. **LCID values:**
   - 1–32: Data LCIDs
   - 0x3D: Short BSR
   - 0x3E: Long BSR
   - 0x3F: Padding

Chapter 12

UE HARQ Management

12.1 HARQ Process Handling

```
void nr_ue_gf_process_harq_feedback(NR_UE_MAC_INST_t *mac,
                                    int harq_pid,
                                    bool ack) {
  // Find GF config using this HARQ process
  nr_gf_config_t *gf = NULL;
  for (int i = 0; i < mac->num_gf_configs; i++) {
    if (mac->gf_config[i].enabled &&
        mac->gf_config[i].harq_process_id == harq_pid) {
      gf = &mac->gf_config[i];
      break;
    }
  }

  if (!gf) return;

  NR_UE_UL_HARQ_INFO_t *harq = &mac->ul_harq_info[harq_pid];

  if (ack) {
    // Success - toggle NDI, reset round
    gf->ndi_toggle = !gf->ndi_toggle;
    harq->round = 0;

    // Free TX buffer
    if (harq->tx_buffer) {
      free(harq->tx_buffer);
      harq->tx_buffer = NULL;
    }

    gf->successful_tx_count++;
  } else {
    // NACK - increment round
    harq->round++;
    gf->harq_nack_count++;

    if (harq->round >= gf->max_retransmissions) {
      // Max retransmissions reached
      gf->ndi_toggle = !gf->ndi_toggle;
      harq->round = 0;

      if (harq->tx_buffer) {
        free(harq->tx_buffer);
        harq->tx_buffer = NULL;
      }
```

```

      gf->failed_tx_count++;
    }
  }
}
```

Listing 12.1: HARQ Feedback Processing (UE)

Part IV

Integration and Deployment

Chapter 13

File Structure

13.1 gNB Files

```
openair2/LAYER2/NR_MAC_gNB/
|-- gNB_scheduler_gf.c       # NEW: Complete GF scheduler implementation
|-- nr_mac_gNB.h             # MODIFIED: Added GF data structures
|-- mac_proto.h              # MODIFIED: Added GF function declarations
|-- gNB_scheduler.c          # MODIFIED: Integration hook
`-- main.c                   # MODIFIED: GF initialization
```

Listing 13.1: gNB File Structure

13.2 UE Files

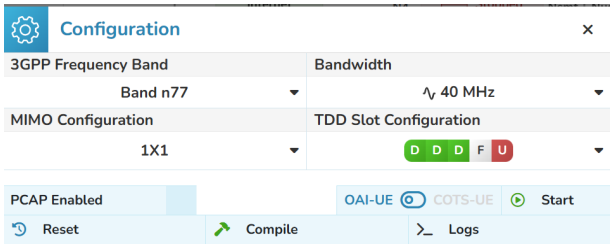
```
openair2/LAYER2/NR_MAC_UE/
|-- nr_ue_scheduler.c        # MODIFIED: Added GF functions
|-- config_ue.c              # MODIFIED: Added GF configuration functions
|-- mac_defs.h               # MODIFIED: Added GF data structures
|-- mac_proto.h              # MODIFIED: Added GF function declarations
`-- main_ue_nr.c             # MODIFIED: GF initialization
```

Listing 13.2: UE File Structure

Chapter 14

Build Integration

14.1 gNB Configuration



14.2 Build Commands for UE

```
clear && sudo ./nr-uesoftmodem -r 106 --numerology 1 --band 77 -C 4019160000 --
    ssb 144 -E --continuous-tx --ue-fo-compensation --usrp-args "serial=326F0F9"
     --ue-txgain 0 --ue-rxgain 76
```

Listing 14.1: Build Command

Chapter 15

Configuration Parameters

15.1 Default Configuration

Table 15.1 shows the default configuration parameters used for testing.

Table 15.1: Default GF Configuration Parameters

Parameter	Value	Description
periodicity	4	GF occasion every 4 slots
offset	0	Start at slot 0
start_symbol	2	Start at symbol 2
nr_of_symbols	12	12 symbols
rb_start	0	Start at PRB 0
rb_size	10	10 PRBs
mcs	9	MCS 9 (QPSK)
mcs_table	0	Table 1
harq_process_id	0	HARQ process 0
dmrs_config_type	0	Type 1
dmrs_ports	1	Port 0

15.2 Configuration Validation

Both UE and gNB perform validation of configuration parameters:

```
// Periodicity must be > 0
if (gf_params->periodicity == 0) return -1;

// Offset must be < periodicity
if (gf_params->offset >= gf_params->periodicity) return -1;

// Symbol allocation must fit in slot
if (gf_params->start_symbol + gf_params->nr_of_symbols > 14) return -1;

// RB allocation must fit in BWP
if (gf_params->rb_start + gf_params->rb_size > bwp_size) return -1;

// MCS must be valid
if (gf_params->mcs > 28) return -1;

// HARQ process must be valid
```

```
if (gf_params->harq_process_id >= 16) return -1;
```

Listing 15.1: Configuration Validation

Chapter 16

Known Limitations and Current problems

16.1 Current Limitations

Table 16.1: Known Limitations

Limitation	Description	Impact
HARQ Feedback (gNB)	Placeholder implementation	gNB doesn't send ACK-/NACK via DCI
VRB Map	GF resources not marked	Potential collision with normal scheduler
Collision Detection	No mechanism	Multiple UEs on same resources
Dynamic Config	Static configuration	No RRC signaling integration
Link Adaptation	Fixed MCS	No adaptive MCS based on channel

16.2 Current Problems

The following issues are currently being investigated:

16.2.1 USRP Operating on USB 2.0 Instead of USB 3.0

The USRP software-defined radio is being detected and operating over USB 2.0 instead of the expected USB 3.0 interface. This results in:

- Reduced throughput capacity
- Potential sample drops and underruns
- Limited bandwidth for higher sample rates

Diagnostic steps:

```
# Check USB connection speed
lsusb -t | grep -i usb

# Verify USRP detection
```

```
uhd_find_devices

# Check for USB 3.0 controller
lspci | grep -i usb
```

Possible causes:

- USB cable not rated for USB 3.0 (SuperSpeed)
- Connected to USB 2.0 port instead of USB 3.0 port
- Missing or outdated USB 3.0 host controller drivers
- BIOS/UEFI USB 3.0 support disabled

16.2.2 Invalid MAC PDU Parsing Error

The gNB is reporting invalid MAC PDU errors during Grant-Free reception:

```
[NR_MAC] Invalid PDU in 980.3 for RNTI 21B4!
        mac_subheader_len: 1, mac_len: 6, remaining pdu_len: 3
```

Analysis:

- `mac_subheader_len: 1` — Fixed subheader detected (1 byte)
- `mac_len: 6` — MAC CE/SDU claims 6 bytes of payload
- `remaining pdu_len: 3` — Only 3 bytes remain in PDU buffer
- **Mismatch:** The subheader indicates 6 bytes but only 3 bytes are available

Possible causes:

- Incorrect subheader type selection (using fixed header when short/long is needed)
- Length field not in network byte order for long subheaders
- TBS mismatch between UE and gNB calculations
- MAC PDU buffer corruption or incorrect pointer arithmetic
- BSR size calculation error (Short BSR is 1 byte payload, not 6)

Recommended investigation:

1. Enable `ENABLE_MAC_PAYLOAD_DEBUG` to dump full MAC PDU hex
2. Verify BSR LCID (0x3D) is followed by exactly 1 byte payload
3. Check that SDU subheaders use correct F-bit (0 for short, 1 for long)
4. Compare TBS values computed on both UE and gNB sides
5. Verify padding subheader (0x3F) is only added once at the end

16.3 Problem Analysis

16.3.1 Original Problematic Implementation

The original implementation had several critical issues that deviated from OAI standards:

```c
// ========== Attach MAC PDU to PUSCH ==========
uint8_t *mac_pdu = (uint8_t *)malloc(TBS);  // Issue 1: malloc usage
memset(mac_pdu, 0, TBS);

int pdu_len = nr_ue_gf_build_mac_pdu(mac, gf, mac_pdu, TBS);

// Issue 2: Custom HARQ structure instead of OAI standard
NR_UE_UL_HARQ_INFO_t *harq = &mac->ul_harq_info[gf->harq_process_id];

if (harq->round == 0 || gf->ndi_toggle != harq->ndi) {
    // Issue 3: Storing MAC PDU in custom buffer
    harq->tx_buffer = mac_pdu;
    harq->tx_buffer_size = TBS;
    harq->ndi = gf->ndi_toggle;
    harq->round = 0;
} else {
    free(mac_pdu);  // Issue 4: Manual memory management
}

// Issue 5: Missing PHY interface attachment
// MAC PDU never reaches PHY layer through proper channels
```

Listing 16.1: Original Problematic Approach

16.3.2 Issues Identified

1. **Memory Management**: Use of `malloc()` instead of OAI's pre-allocated buffers
2. **HARQ Structure**: Custom `NR_UE_UL_HARQ_INFO_t` instead of standard OAI structures
3. **PHY Interface**: MAC PDU not attached to `tx_request_body.fapiTxPdu`
4. **Buffer Management**: Manual memory management instead of OAI patterns
5. **Integration**: Poor integration with existing OAI HARQ mechanisms

16.4 OAI Standard Approach

16.4.1 How Normal OAI Handles MAC PDU Transmission

Based on analysis of `nr_ue_scheduler.c`, the standard OAI flow follows this pattern:

```c
// 1. Use pre-allocated buffer
uint8_t ulsch_input_buffer[MAX_ULSCH_PAYLOAD_BYTES];

// 2. Build MAC PDU directly into buffer
nr_ue_get_sdu(mac, cc_id, frame_tx, slot_tx, gNB_index,
              ulsch_input_buffer, TBS_bytes, tx_power, P_CMAX, &BSRsent);

// 3. Attach to ul_config PDU (CRITICAL OAI INTERFACE)
pdu->tx_request_body.fapiTxPdu = ulsch_input_buffer;
pdu->tx_request_body.pdu_length = TBS_bytes;

// 4. Release ul_config (makes PDU available to PHY)
release_ul_config(pdu, true);
```

Listing 16.2: Standard OAI MAC PDU Flow

16.4.2 Key OAI Principles

- **Pre-allocated Buffers**: No dynamic allocation during runtime

- **Standard Interfaces**: Use `tx_request_body.fapiTxPdu` for PHY communication
- **Integrated HARQ**: Leverage existing OAI HARQ management structures
- **Memory Safety**: Avoid manual memory management

16.5 Corrected Implementation

16.5.1 Corrected MAC PDU Attachment

The following implementation aligns with OAI standards and resolves the identified issues:

```
// ========== Attach MAC PDU to PUSCH ==========

// Use pre-allocated buffer (OAI standard approach)
uint8_t ulsch_input_buffer[MAX_ULSCH_PAYLOAD_BYTES];
memset(ulsch_input_buffer, 0, TBS);

// Build MAC PDU directly into OAI buffer
int pdu_len = nr_ue_gf_build_mac_pdu(mac, gf, ulsch_input_buffer, TBS);

if (pdu_len <= 0) {
    LOG_W(NR_MAC, "[GF] MAC PDU build failed, filling with padding\n");
    memset(ulsch_input_buffer, 0, TBS);
    ulsch_input_buffer[0] = UL_SCH_LCID_PADDING;
    pdu_len = TBS;
}

// Attach MAC PDU to ul_config PDU (OAI standard interface)
pusch_pdu->tx_request_body.fapiTxPdu = ulsch_input_buffer;
pusch_pdu->tx_request_body.pdu_length = pdu_len;

// Set HARQ parameters in PUSCH PDU (OAI standard fields)
pusch_pdu->pusch_data.harq_process_id = gf->harq_process_id;
pusch_pdu->pusch_data.new_data_indicator = gf->ndi_toggle;
pusch_pdu->pusch_data.rv_index = gf->rv_sequence[0];

// Update OAI's standard HARQ tracking
NR_UE_sched_ctrl_t *sched_ctrl = &mac->sched_ctrl;
if (sched_ctrl && gf->harq_process_id < NR_MAX_HARQ_PROCESSES) {
    NR_UE_ul_harq_t *harq = &sched_ctrl->ul_harq_processes[gf->harq_process_id];

    // Handle new transmission vs retransmission
    if (pusch_pdu->pusch_data.new_data_indicator != harq->ndi) {
        // New transmission
        harq->ndi = pusch_pdu->pusch_data.new_data_indicator;
        harq->round = 0;

        LOG_I(NR_MAC, "[GF] New transmission: HARQ pid=%d, NDI=%d, TBS=%d\n",
              gf->harq_process_id, harq->ndi, TBS);
    } else {
        // Retransmission
        harq->round++;
        pusch_pdu->pusch_data.rv_index = gf->rv_sequence[harq->round % 4];

        LOG_I(NR_MAC, "[GF] Retransmission: HARQ pid=%d, round=%d, RV=%d\n",
              gf->harq_process_id, harq->round,
              pusch_pdu->pusch_data.rv_index);
    }
}
```

Listing 16.3: Corrected Implementation

16.5.2 Release ul_config

```
// ========== Release ul_config ==========
release_ul_config(pdu, true);  // true = PDU is valid and ready for PHY

// Update grant-free statistics
gf->tx_count++;
gf->last_tx_frame = frame;
gf->last_tx_slot = slot;

LOG_I(NR_MAC, "[UE %d] GF PUSCH scheduled at %d.%d: TBS=%d, HARQ=%d, RV=%d\n",
      mac->ue_id, frame, slot, TBS,
      pusch_pdu->pusch_data.harq_process_id,
      pusch_pdu->pusch_data.rv_index);
```

Listing 16.4: Release ul_config

16.6 Expected Results

16.6.1 Resolution of Issues

After implementing the corrected approach, the following improvements are expected:

1. **gNB Parsing**: Elimination of "Invalid PDU" errors
2. **MAC PDU Delivery**: Proper transmission through OAI's PHY interface

Note that this current implementation has not been tested properly.